

Exercise 8

Implementing Classes 3: Pokemon Class Hierarchy

Introduction:

When the code base of a problem will go in size so will its complexity. But when we think of the entire program as a single entity, composed of autonomous sub-entities, then we can simplify its complexity. In computer science, abstracting information for the perusal of the user is the first step in describing the features of a program. The class in C++ is the means in which the abstraction can be translated into a user-defined type.

For this exercise, a Pokemon would be represented as an object using class inheritance. A base class Pokemon defines the shared attributes of all Pokemon, while derived classes model specific types. Each Pokemon type inherits the common properties of the base class and extends them with type-specific attributes and behaviors. The relationship between base and derived classes follows the is-a principle: `DerivedClass : public BaseClass`.

Learning Outcomes:

- Create a base class Pokemon and derive specialized subclasses that inherit and extend its member variables and functions.
- Implement and test class inheritance in C++ by creating instances of the derived classes and invoking both inherited and overridden functions.

Problem Background

1. Create a project file for this exercise.

Base Class: Pokemon

2. On a header file, `pokemon.h`, create a base class named `Pokemon` with the following private variables:

```
string  name;           // the name of the Pokemon
int     level;          // the current level of the Pokemon (1-100)
int     hp;             // current hit points of the Pokemon
int     maxHp;          // maximum hit points of the Pokemon
string  type;           // the elemental type of the Pokemon
```

3. In the base class, write the function prototypes as indicated below:

```
Pokemon();
    // default constructor, sets numerics to 0
Pokemon(string, int, int);
    // parameterized: name, level, maxHp
void setName(string);           // set the name of the Pokemon
```

```

void setLevel(int);           // set the level of the Pokemon
void setHp(int);
    // set the current HP of the Pokemon
void setType(string);        // set the elemental type
string getName();
    // return the name of the Pokemon
int getLevel();
    // return the level of the Pokemon
int getHp();                 // return the current HP
int getMaxHp();              // return the maximum HP
string getType();            // return the elemental type
bool isAlive();
    // return true if current HP is above 0
~Pokemon();
    // destructor, no content on function body
virtual void attack(Pokemon&);
/* performs a basic attack on another Pokemon,
   reducing its HP; overridden in derived classes. */

void displayStats();
/* displays the name, type, level, and HP of
   the Pokemon to standard output. */

```

4. On another file `pokemonImp.cpp`, implement the function body for the class `Pokemon`.

Derived Class 1: `FirePokemon` (Fire Type)

5. On a header file, `firePokemon.h`, declare a class `FirePokemon` that inherits publicly from `Pokemon`. Add the following private variables specific to the Fire type:

```

int      burnChance;  // burn probability on attack (0-100)
double   firePower;   // multiplier for fire-type move damage

```

6. In the `FirePokemon` class, write the function prototypes as indicated below:

```

FirePokemon();
    // default constructor, calls Pokemon()
FirePokemon(string, int, int);
    // parameterized: name, level, maxHp
void setBurnChance(int);
    // set the burn chance percentage
void setFirePower(double);
    // set the fire power multiplier
int getBurnChance();
    // return the burn chance percentage

```

```

double getFirePower();
    // return the fire power multiplier
~FirePokemon();
    // destructor, no content on function body
void attack(Pokemon&);
/* overrides base attack; applies firePower multiplier
    and has burnChance probability of inflicting burn. */

void useFlamethrower(Pokemon&);
/* powerful fire move dealing double fire damage;
    always attempts to inflict the burn status. */

```

7. On another file firePokemonImp.cpp, implement the function body for the class FirePokemon.

Derived Class 2: WaterPokemon (Water Type)

8. On a header file, waterPokemon.h, declare a class WaterPokemon that inherits publicly from Pokemon. Add the following private variables specific to the Water type:

```

int      waterPressure; // pressure level of water moves (1-10)
bool     canDive;       // true if the Pokemon can use Dive

```

9. In the WaterPokemon class, write the function prototypes as indicated below:

```

WaterPokemon();
    // default constructor, calls Pokemon()
WaterPokemon(string, int, int);
    // parameterized: name, level, maxHp
void setWaterPressure(int);      // set the water pressure level
void setCanDive(bool);
    // set whether the Pokemon can dive
int getWaterPressure();
    // return the water pressure level
bool getCanDive();
    // return whether the Pokemon can dive
~WaterPokemon();
    // destructor, no content on function body
void attack(Pokemon&);
/* overrides base attack; scales damage by waterPressure
    and gains a bonus against Fire-type opponents. */

void useHydroBlast(Pokemon&);
/* high-pressure move with damage scaled to waterPressure;
    if canDive is true, adds an evasion bonus. */

```

10. On another file `waterPokemonImp.cpp`, implement the function body for the class `WaterPokemon`.

Derived Class 3: `GrassPokemon` (Grass Type)

11. On a header file, `grassPokemon.h`, declare a class `GrassPokemon` that inherits publicly from `Pokemon`. Add the following private variables specific to the Grass type:

```
int      healAmount;    // HP restored per turn by regeneration
double   poisonChance; // probability to poison opponent (0.0-1.0)
```

12. In the `GrassPokemon` class, write the function prototypes as indicated below:

```
GrassPokemon();
    // default constructor, calls Pokemon()
GrassPokemon(string, int, int);
    // parameterized: name, level, maxHp
void setHealAmount(int);           // set the passive heal amount
void setPoisonChance(double);
    // set the poison chance probability
int getHealAmount();
    // return the passive heal amount
double getPoisonChance();
    // return the poison chance probability
~GrassPokemon();
    // destructor, no content on function body
void attack(Pokemon&);
/* overrides base attack; has poisonChance probability
   of poisoning the opponent, reducing HP per turn. */

void regenerate();
/* restores HP equal to healAmount, not exceeding maxHp;
   displays the amount healed to standard output. */
```

13. On another file `grassPokemonImp.cpp`, implement the function body for the class `GrassPokemon`.

14. Test the operation of all classes on `main.cpp`. Create at least one instance of each derived class, call `displayStats()`, and simulate an attack sequence between `Pokemon` of different types using the overridden `attack()` functions.